

EEL4712 Digital Design

Lecture 28 – System Verilog Assertions II

Rickard Ewetz
Spring 2026

Agenda

- Review of last lecture
- Cycle, range, and infinite delay
- Sequences & actions
- Assertions for pipelines
- Assertions for designs with variable delay using queue's

Review of Previous Lecture

- Assertions
  - Immediate assertions
  - Concurrent assertions
- Syntax
  - Assert
  - Property
  - @(posedge clk)
  - Disable iff (rst)
- Implication operator
  - Same cycle: I->
  - Next cycle: I=>

```
assert property (@(posedge clk) disable iff (rst)
    antecedent |-> consequent);
```

- Syntax II
  - $past
  - $stable
  - $rose
  - $fell

Cycle Delay – The ## operator

- Purpose
  - Used in SVA to specify clock cycle delays
  - Allows you to describe temporal relationships between events
  - Interpreted relative to the sampling clock (e.g., @(posedge clk))
- Basic Syntax
  - `##N` → delay of N clock cycles
- Example: `a ##2 b`
  - a must occur
  - 2 clock cycles later
  - b must occur

Example 1

```
assert property (@(posedge clk)
        req |-> ##1 ack
);
```

Example 2

```
assert property (@(posedge clk)
    start |-> ##3 done
);
```

Cycle Delay – Range and Infinite Delay (##)

- Delay Range
  - ##[m:n]
  - Event must occur between m and n cycles later

⑩ Infinite Upper Bound
  - ##[m:$]
  - Event must occur at least m cycles later
  - $ means no upper limit

```
assert property (@(posedge clk)
    $rose(req) |-> ##[1:3] ack);
```

ack must occur **1–3 cycles after req**

```
assert property (@(posedge clk)
    start |-> ##[2:$] done
);
```

done occurs **2 or more cycles later**

What is the limitation of infinite delay?

Example using Range

- When a request is issued:
  - A response must occur **between 2 and 5 cycles later,** which is indicted by setting **resp_valid** to **1**
  - The response data must equal the **sum** of the inputs **a** and **b** from the request cycle

```
assert property (@(posedge clk) disable iff (rst)
    $rose(req) |-> ##[2:5] (resp_valid &&  resp_data == $past(a + b,2))
);
```

Does this work? Do we know the sum is from the correct inputs?

Sequences in SystemVerilog Assertions

Will not be on labs or exams

- Purpose
  - A sequence describes an ordered pattern of events over time
  - Used to specify temporal behavior in designs
  - Often used as building blocks for assertions and properties
- Basic Syntax

```
sequence seq_name;
   sequence_expression;
endsequence
```

- Example
  - `req` occurs
  - **1 cycle later**
  - `ack` occurs

```
sequence req_ack_seq;
      req ##1 ack;
endsequence
```

Sequence with Action in SystemVerilog Assertions

- Purpose
  - Capture values **when an event occurs**
  - Use those values **later in the assertion**
  - Useful when **latency is variable**
- Syntax
  - `(sequence_expression, action)`
  - `sequence_expression` → event that triggers the sequence
  - `action` → assignment executed when the sequence matches

Will not be on labs or exams

(req, sum = a + b)

Example with Sequence and Action

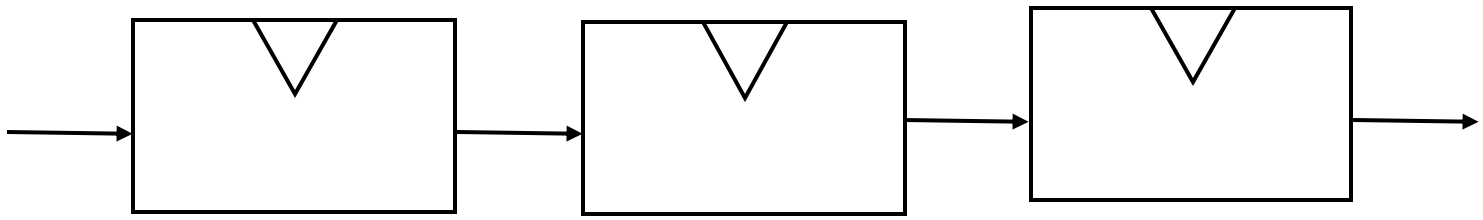
- When a request is issued:
  - A response must occur **between 2 and 5 cycles later,** which is indicted by setting **resp_valid** to **1**
  - The response data must equal the **sum** of the inputs **a** and **b** from the request cycle

```
assert property (@(posedge clk) disable iff (rst)
        (req, sum = a + b) |-> ##[2:5] (resp_valid && resp_data == sum)
);
```

Will not be on labs or exams

Assertions for Pipelines

- Structure
  - Many designs can be viewed as pipelines
  - A delay chain can be viewed as a representative example of a pipeline
- Challenges
  - Pipelines with enable have "variable" latency
  - After reset, it takes CYCLES time to fill up the pipeline with new data



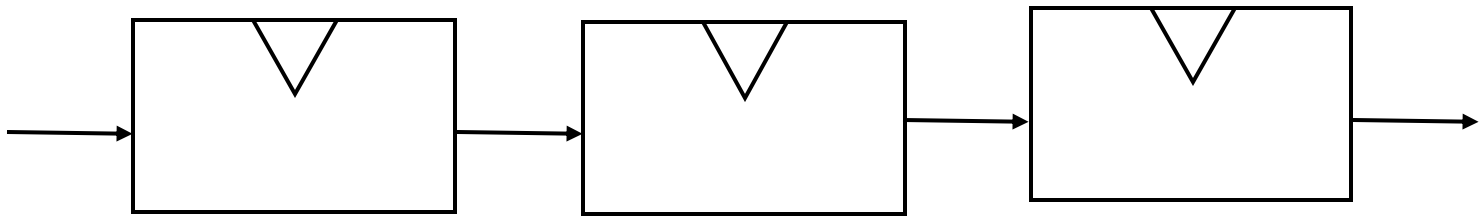
Assertions for Pipelines

- Solution I
  - Use a counter to measure the saturation of the pipeline (this handles the reset challenge)

```
   int count;
always_ff @(posedge clk or posedge rst)
    if (rst) count <= 0;
    else if (count < CYCLES) count <= count + 1;

// Don't check for the output matching the delayed input until an input
// has reached the output.
assert property (@(posedge clk) disable iff (rst || count < CYCLES)
    data_out == $past(data_in, CYCLES));

// Check for correct outputs during reset and until inputs reach the output.
assert property (@(posedge clk) disable iff (count == CYCLES) data_out == '0);
```

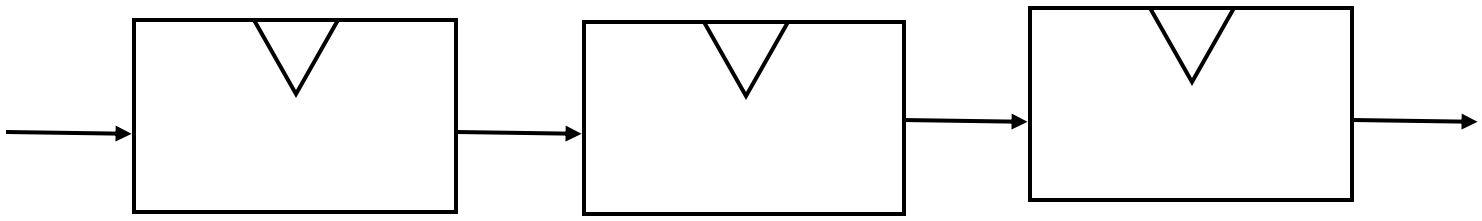


Assertions for Pipelines

- Solution II - Enable
  - Use $past(in, X, en) which queries the value of in @ X cycles back while only counting cycles where enable was high (very useful feature)

```
int count;
always_ff @(posedge clk or posedge rst)
    if (rst) count <= 0;
    else if (en && count < CYCLES) count <= count + 1;


assert property (@(posedge clk) disable iff (rst) count == CYCLES |-> data_out ==
    $past(data_in, CYCLES, en));
```



Limitations of Outlined Approach

- When does this approach not work?
  - Variable latency pipelines
  - Pipelines that reorder elements or drop elements

Assertions for Variable Latency Designs using Queues

- A FIFO is an example of a design with variable latency
- It is difficult to write assertions for FIFOs using the techniques that we have covered so far
- Queues are a good method for handling designs with variable delay from the input to output
  - Implement a reference model for the design using a queue
  - Add elements to the queue when they enter the design
  - Pop() elements from the queue when they leave the designs
  - The reference model should be independent of the design (don't use full or empty signals from the design to check other behaviors)

In-Class Demo

- Design Under Test:
  - `delay.sv` as a representative pipeline
- Testbench Variants for Pipelines using a Delay Chain:
  - Testbench 1: Naïve implementation that mirrors the DUT
    - Significant amount of implementation and risk for errors
  - Testbench 2: Queue-based modeling approach
  - Testbench 3: Reset-aware design using counters
  - Testbench 4: Handling both reset and enable conditions